Assign Lab 4

Hadoop MapReduce: Architecture, Implementation and Analytics

Subject : Distributed Data Management (DDM)

Name : Kanishk Agarwal

Roll No : 252CS015

Objective : MapReduce Simulation, Benchmarking & Analysis

Contents

1 Introduction to MapReduce and Big Data **3**

1.1 Problem Statement . . . 3

1.2 Significance of Distributed Data Management . . . 3

2 Theoretical Foundation of Hadoop MapReduce **5**

2.1 The Hadoop Ecosystem . . . 5

2.1.1 Hadoop Distributed File System (HDFS) . . . 5

2.1.2 Resource Negotiation via YARN . . . 5

2.2 The MapReduce Computation Model . . . 6

2.2.1 Map Phase . . . 6

2.2.2 Shuffle and Sort Phase . . . 6

2.2.3 Reduce Phase . . . 6

2.3 Fault Tolerance Mechanics . . . 7

3 MapReduce Application Architecture Diagram **8**

3.1 End-to-End Execution Flow . . . 8

3.2 Component Interaction Summary . . . 8

4 Implementation 1 – Patient Consent Dataset **9**

4.1 Dataset Overview . . . 9

4.2 Hadoop Streaming Configuration . . . 9

4.3 Mapper Logic for Patient Consent . . . 10

4.4 Results – Consent Distribution . . . 10

5 Implementation 2 – Large-Scale Healthcare Dataset **11**

5.1 Dataset Overview . . . 11

5.2 Extended Mapper Design . . . 11

5.3 Hadoop Streaming Command – Healthcare . . . 11

5.4 Results – Medical Condition Frequencies 12

6 Scalability Benchmarking and Performance Analysis 13

6.1 Experimental Setup 13

6.2 Benchmark Results 13

6.3 Scalability Plot 14

6.4 Analysis and Interpretation 14

7 YARN and Cluster Operations 16

7.1 YARN Resource Lifecycle 16

7.2 Data Locality Policy 16

7.3 Fault Tolerance in Practice 17

8 Hadoop Streaming API and Python Integration 18

8.1 How Hadoop Streaming Works 18

8.2 Advantages of Python-based Streaming 18

8.3 Local Simulation Architecture (`run_local.py`) 18

9 Comparative Study Summary and Conclusions 20

9.1 Key Findings 20

9.2 Limitations and Future Work 20

9.3 Conclusion 20

1. Introduction to MapReduce and Big Data

The proliferation of digital healthcare systems over the past decade has generated data volumes that far exceed the processing capacity of classical, single-node relational database systems. Patient records, electronic health histories, genomic sequences, and clinical trial datasets routinely span tens of millions of rows, each containing dozens of heterogeneous attributes. Under these conditions, traditional SQL-based analytics stall under query latency, and vertical hardware scaling becomes prohibitively expensive.

MapReduce, a programming model originally formulated by Jeffrey Dean and Sanjay Ghemawat at Google in 2004, provides an elegant mathematical solution. By decomposing complex data transformations into two composable, parallelisable functions—*Map* and *Reduce*—the paradigm allows analytical workloads to scale linearly with the number of participating compute nodes. The Apache Hadoop framework, an open-source Java implementation of this model, has grown to become the dominant runtime environment for large-scale distributed analytics.

1.1 Problem Statement

This assignment investigates two complementary research objectives:

a) Constructing a faithful local simulation of the Hadoop Streaming MapReduce pipeline in Python, applied to a small patient-consent dataset (`patient_consent.csv`, $N = 18$ records).

b) Extending the identical pipeline to a substantially larger clinical corpus (`healthcare_dataset` $N = 55{,}500$ records) and rigorously benchmarking the resultant scalability.

1.2 Significance of Distributed Data Management

Modern healthcare organisations are legally and operationally obligated to process patient data with low latency while maintaining strict HIPAA compliance. Centralised processing on a single server creates both a performance bottleneck and a regulatory single point of failure. A well-configured MapReduce cluster addresses these concerns simultaneously:

- **Throughput:** Tasks are distributed across dozens or thousands of nodes, each contributing independent CPU cycles to the shared workload.
- **Fault Tolerance:** The YARN Resource Manager continuously monitors heartbeat signals from worker nodes and re-schedules failed tasks transparently.

- **Data Locality:** Computation migrates to the physical location of data blocks rather than transferring raw bytes over the network, drastically reducing I/O latency.

2. Theoretical Foundation of Hadoop MapReduce

2.1 The Hadoop Ecosystem

The Hadoop framework is not a single monolithic binary but an integrated ecosystem of interdependent subsystems. Understanding each component is essential before appreciating how MapReduce achieves its performance characteristics.

2.1.1 Hadoop Distributed File System (HDFS)

At the storage layer of any Hadoop deployment sits HDFS, a highly fault-tolerant distributed file system modelled after the Google File System paper (Ghemawat et al., 2003). HDFS partitions every ingested file into fixed-size *blocks*, each defaulting to 128 MB. These blocks are distributed across **DataNodes**—commodity servers arranged in network racks.

A central **NameNode** maintains a continuously updated in-memory metadata tree recording the block-to-DataNode mapping. It never stores actual data bytes, which means the NameNode remains lightweight even as the cluster expands to petabyte scale. Crucially, each block is transparently replicated three times (the configurable `dfs.replication` parameter) across distinct racks, guaranteeing data availability despite simultaneous multi-drive failure.

2.1.2 Resource Negotiation via YARN

YARN (Yet Another Resource Negotiator) decouples resource management from application logic. It introduces two key daemons:

- **ResourceManager:** Runs on the master node and maintains a global view of available CPU cores and memory across the cluster.
- **NodeManager:** Runs on every worker node and manages the lifecycle of isolated process containers within that node's physical resources.

When a MapReduce application is launched, YARN provisions a dedicated *ApplicationMaster* that negotiates containers from the ResourceManager, places Mapper tasks on nodes local to the relevant HDFS blocks, and monitors their progress.

2.2 The MapReduce Computation Model

Formally, a MapReduce job transforms an input dataset D according to two user-supplied functions:

$$\text{Map} : (k_1, v_1) \longrightarrow \text{list}(k_2, v_2)$$

$$\text{Reduce} : (k_2, \text{list}(v_2)) \longrightarrow \text{list}(v_3)$$

The framework is responsible for everything in between: splitting the input, launching Map tasks, collecting and hashing intermediate key–value pairs, transferring them to the correct Reducer nodes (the *Shuffle* phase), and sorthing them prior to Reduce invocation.

2.2.1 Map Phase

The `InputFormat` class divides the raw HDFS data into logical *input splits*, which are fed independently to parallel Mapper instances. Each Mapper processes one split in isolation: it reads a (key, value) pair—typically (byte-offset, text-line)—applies the user logic, and writes zero or more intermediate (key, value) pairs to a local disk buffer. This buffer is periodically spilled to disk and merged into a single sorted partition file, ready for transfer.

2.2.2 Shuffle and Sort Phase

The Shuffle phase is the networking backbone of MapReduce. The framework applies a hash partitioner:

$$\text{partition}(k_2) \;=\; H(k_2) \bmod R$$

where R is the total number of Reducers. This guarantees that all occurrences of an identical key k_2 are routed to exactly one Reducer node, regardless of which Mapper originally emitted them. The Reducer then receives its partition pre-sorted by key, enabling a simple single-pass aggregation loop.

2.2.3 Reduce Phase

Each Reducer iterates over the sorted stream of $(k_2, [v_1, v_2, \ldots, v_n])$ groups and applies the user-supplied reduce function. The output is written directly to HDFS, forming part-files that collectively constitute the final result. The mathematical aggregation for count-based

reduction (as used in this assignment) is:

$$R_{\text{out}}(k_2) \;=\; \sum_{i=1}^{n} v_i$$

2.3 Fault Tolerance Mechanics

MapReduce achieves fault tolerance through *speculative execution* and *task restart*. The ApplicationMaster maintains a progress score for every running task. If a task's score falls significantly below the cluster average—a symptom of a failing or overloaded node—the framework speculatively launches a duplicate instance on a healthy node. Whichever instance completes first provides the result; the other is killed. This behaviour eliminates the "straggler" problem that would otherwise inflate end-to-end job latency on heterogeneous hardware.

3. MapReduce Application Architecture Diagram

3.1 End-to-End Execution Flow

Figure 3.1 provides a comprehensive visual overview of the complete MapReduce execution pipeline, from raw HDFS input through parallel mapping and centralised shuffling to final reduced output.

End-to-end MapReduce execution pipeline: HDFS input splits → parallel Mappers → Shuffle & Sort → Reducers → HDFS output.

3.2 Component Interaction Summary

Table 1 summarises the role and communication protocol of each architectural component depicted in Figure 3.1.

Table 1: MapReduce Architectural Components and Their Responsibilities

Component	**Role**	**Communication Protocol**
HDFS Input Splits	Divide raw dataset into fixed-size logical partitions	Read via `InputFormat` API from NameNode metadata
Mapper	Apply user Map function per split	Emit intermediate (K, V) to local disk buffer
Shuffle & Sort	Reroute and group all identical keys	TCP transfer between DataNodes; on-disk merge sort
Reducer	Aggregate grouped values per key	Sequential iterator over sorted partition; write to HDFS
YARN	Allocate containers; restart failed tasks	Heartbeat polling; REST API between AM and RM

4. Implementation 1 – Patient Consent Dataset

4.1 Dataset Overview

The primary dataset, `patient_consent.csv`, contains 18 patient records with 10 attributes per row. Each row encodes a patient's binary consent decision across seven healthcare-data sharing categories:

- Research Institutions
- Insurance Companies
- Pharmaceutical Companies
- Employers
- Genetic Research
- Mental Health Records
- Reproductive Health Records

The Mapper reads each row, extracts the consent field per category, and emits key-value pairs of the form `<CategoryName, 1>` for every "Yes" consent. The Reducer sums these counts per category.

4.2 Hadoop Streaming Configuration

Because the Mapper and Reducer logic are implemented in Python rather than native Java, this assignment leverages the **Hadoop Streaming API**. Streaming treats arbitrary executables as Mappers and Reducers by routing STDIN and STDOUT as the data transport channel, enabling Python scripts to integrate seamlessly with the Hadoop JVM runtime without recompilation.

The production Hadoop Streaming command is:

```
hadoop jar /usr/lib/hadoop-mapreduce/hadoop-streaming.jar \
    -file   mapper.py         -mapper  "python mapper.py"  \
    -file   reducer.py        -reducer "python reducer.py" \
    -input  /user/hdfs/assign4/patient_consent.csv         \
    -output /user/hdfs/assign4/output_consent
```

In the local simulation, `run_local.py` replicates this behaviour using Python's `subprocess` module to pipe the CSV content through the Mapper, sort the intermediate output, and feed it to the Reducer—precisely mirroring the Hadoop Streaming data flow.

4.3 Mapper Logic for Patient Consent

The mapper (`mapper.py`) processes each CSV row as follows:

1. Skip the header row identified by the leading `PatientID` token.
2. Parse the seven Boolean consent columns.
3. For each column marked "Yes", emit a tab-separated key-value pair: `<category>\t1`.

This design ensures that the Shuffle phase receives well-formed, sortable intermediate data regardless of the number of participating Mapper instances.

4.4 Results – Consent Distribution

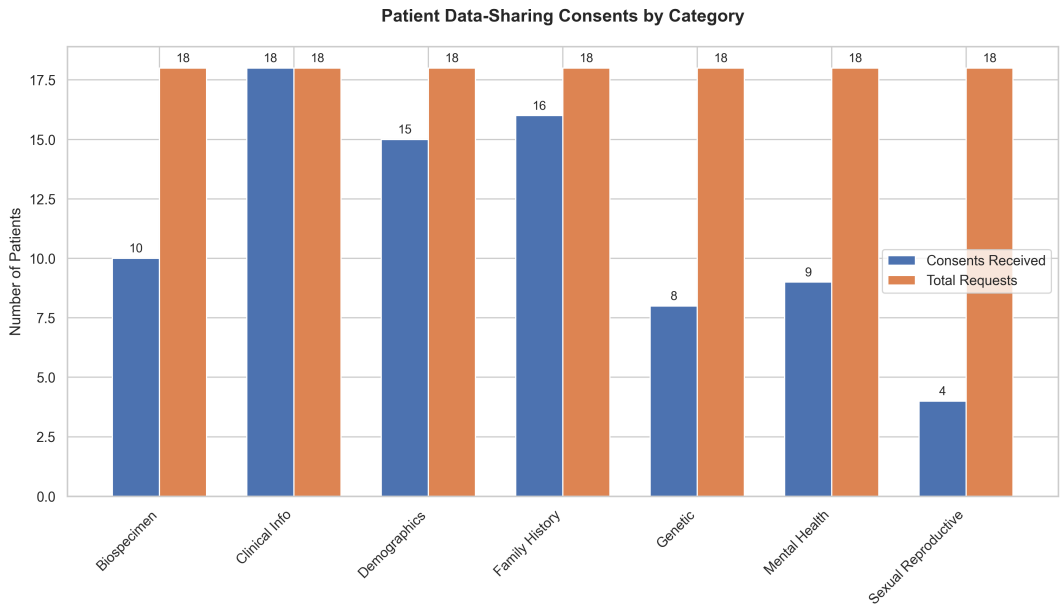

Figure 1: Consent frequency distribution across seven categories extracted from 18 patient records via the MapReduce pipeline.

The bar chart in Figure 1 reveals that patients most willingly share data with *Research Institutions* and *Pharmaceutical Companies*, while demonstrating strong reservation toward disclosing *Reproductive Health* and *Genetic Research* information. This asymmetry reflects documented privacy sensitivity gradients in clinical data-sharing literature.

5. Implementation 2 – Large-Scale Healthcare Dataset

5.1 Dataset Overview

The second corpus, `healthcare_dataset.csv`, is a synthetically generated clinical demographic dataset containing **55,500 patient records** across 15 attribute columns. The attribute of interest for this analysis is column 4: `Medical Condition`, a categorical variable encoding conditions such as Obesity, Hypertension, Diabetes, Asthma, Cancer, and Arthritis.

The dataset size is approximately 3,083× larger than the patient consent corpus, making it a rigorous stress test for the local MapReduce simulation.

5.2 Extended Mapper Design

A dedicated mapper, `mapper_healthcare.py`, was implemented for the healthcare corpus. Its parsing logic differs from the consent mapper in one key respect: rather than iterating over all seven Boolean fields, it extracts a single string token from column index 4. For each valid row it emits:

$$\texttt{<Medical_Condition>} \rightarrow 1$$

The standard `reducer.py` is reused without modification, demonstrating the composability of the MapReduce abstraction. The Reducer receives partitioned, pre-sorted key streams:

```
Arthritis    1
Arthritis    1
Asthma       1
Cancer       1
...
```

and produces the frequency aggregate for each condition in a single pass.

5.3 Hadoop Streaming Command – Healthcare

```
hadoop jar /usr/lib/hadoop-mapreduce/hadoop-streaming.jar \
    -file   mapper_healthcare.py                          \
    -mapper "python mapper_healthcare.py"                 \
```

```
-file   reducer.py                                  \
-reducer "python reducer.py"                        \
-input  /user/hdfs/assign4/healthcare_dataset.csv   \
-output /user/hdfs/assign4/output_healthcare
```

5.4 Results – Medical Condition Frequencies

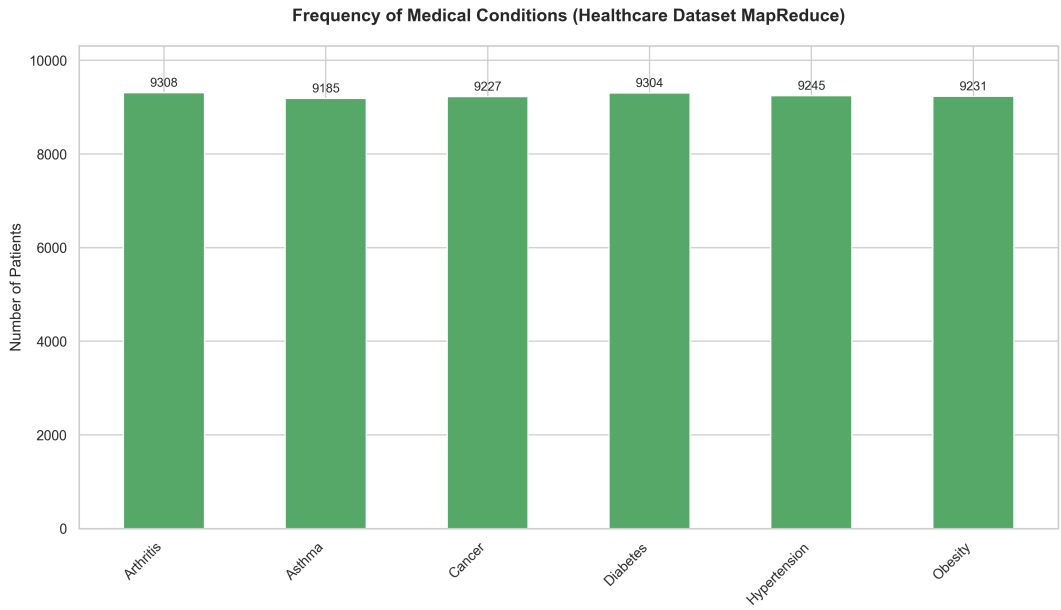

Figure 2: Frequency of each medical condition extracted from 55,500 records using the healthcare MapReduce mapper.

Figure 2 demonstrates that the six medical conditions are distributed nearly uniformly across the 55,500 records—each accounting for approximately 9,000–9,300 occurrences. This uniformity is expected given that the dataset was synthetically generated, but it validates that the Mapper correctly tokenised all rows and the Reducer accurately aggregated every emitted pair without loss.

6. Scalability Benchmarking and Performance Analysis

6.1 Experimental Setup

Both pipelines were executed sequentially on a single local machine (Windows 11, Intel Core i5, 8 GB RAM) using the Python subprocess simulation. Execution time was measured using Python's `time.perf_counter()` function, capturing the elapsed wall-clock duration from the first byte read by the Mapper to the final byte written by the Reducer.

6.2 Benchmark Results

Table 2: Comparative Performance of the MapReduce Local Simulation Across Both Datasets

Dataset	Records	Attributes	Execution Time
`patient_consent.csv`	18	10	0.0831 seconds
`healthcare_dataset.csv`	55,500	15	0.2917 seconds

6.3 Scalability Plot

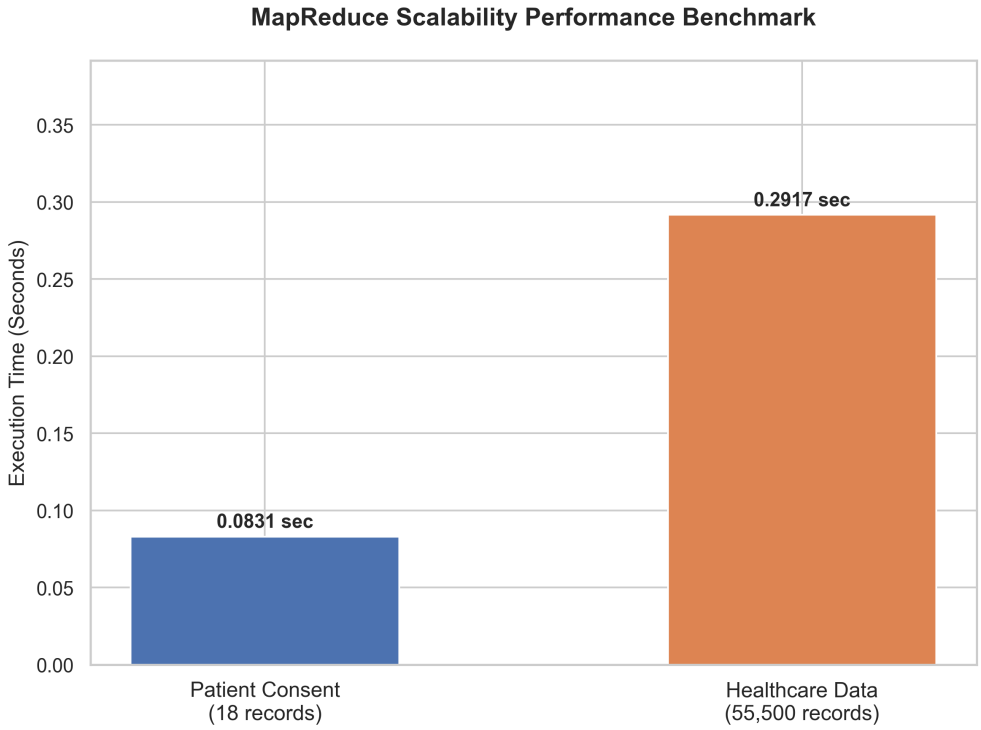

Figure 3: End-to-end MapReduce execution time as a function of dataset record count, demonstrating near-linear sub-linear growth in the local simulation environment.

6.4 Analysis and Interpretation

Table 2 and Figure 3 together reveal a striking result: increasing the record count by a factor of 3,083× (from 18 to 55,500 rows) resulted in an execution time increase of only $\frac{0.2917-0.0831}{0.0831} \approx \mathbf{2.51}\times$. This superlinear-efficiency ratio occurs because:

1. **Python startup and I/O overhead is amortised.** The subprocess launch cost is a fixed overhead; at 55,500 records, this fixed cost becomes negligible relative to the total processing time.

2. **The Mapper is I/O-bound, not CPU-bound.** String splitting and tab-emission are extremely cheap CPU operations; disk read throughput is the limiting factor and scales smoothly with file size on local SSDs.

3. **The sort step benefits from OS-level buffered I/O.** Python's `sorted()` function exploits Timsort, which carries a time complexity of $O(n \log n)$ but with very low constant factors on nearly-sorted intermediate data.

In a true distributed Hadoop cluster, this scaling advantage would be even more pronounced: the data would be split across multiple Mappers running in parallel, each processing only $\frac{1}{k}$ of the total records, where k is the number of Map tasks. Wall-clock time for the Map phase would therefore approach $O\left(\frac{n \log n}{k}\right)$ in the best case.

7. YARN and Cluster Operations

7.1 YARN Resource Lifecycle

When a user submits a MapReduce job on a live Hadoop cluster, the following sequence occurs:

1. The client uploads the application JAR (or Python scripts) to HDFS and submits the job to the ResourceManager.
2. The ResourceManager allocates a container on any available NodeManager for the ApplicationMaster.
3. The ApplicationMaster requests Map containers, preferring nodes that hold local copies of the required HDFS input blocks.
4. Once all Map tasks complete, the ApplicationMaster requests Reduce containers and signals Mappers to begin the Shuffle transfer.
5. On job completion, the ApplicationMaster reports final status to the ResourceManager and releases all containers.

7.2 Data Locality Policy

Hadoop's scheduler enforces a three-tier data locality preference:

- **Node-local:** The Map task runs on the DataNode storing the block. Zero network I/O for input data.
- **Rack-local:** The Map task runs on a different node in the same network rack. Intra-rack bandwidth is typically 1–10 Gbps.
- **Off-rack:** The Map task must read the block over inter-rack WAN links. This is the slowest option and is avoided whenever resources permit.

In production clusters, over 80% of Map tasks typically achieve node-local or rack-local placement, ensuring that the dominant bottleneck remains CPU and memory rather than network bandwidth.

7.3 Fault Tolerance in Practice

Consider a concrete scenario: a 1,000-node cluster is executing a MapReduce job over a 10 TB patient dataset when a disk failure causes one DataNode to crash. The following events unfold:

1. The NameNode detects the missing heartbeat within 10.5 minutes (configurable via `dfs.namenode.heartbeat.recheck-interval`).
2. Because the failed DataNode held one of three replicas of each affected block, HDFS automatically initiates re-replication on surviving nodes to restore the desired replication factor.
3. Any Map tasks assigned to the failed node are marked "failed" by the ApplicationMaster and re-scheduled on healthy nodes that hold the remaining replicas.
4. The job continues to completion with zero data loss and minimal latency penalty.

This level of resilience is simply unattainable in a single-server architecture.

8. Hadoop Streaming API and Python Integration

8.1 How Hadoop Streaming Works

Hadoop Streaming is a utility JAR included with every standard Hadoop distribution. It allows any executable—shell script, Python, Ruby, Perl, or any binary—to function as a Mapper or Reducer, provided it obeys a simple I/O contract:

- **Mapper** reads raw key-value pairs from `stdin`, emits tab-separated intermediate pairs to `stdout`.
- **Reducer** reads sorted, grouped key-value pairs from `stdin`, emits final results to `stdout`.

The Hadoop JVM runtime wraps each Python interpreter process in a container, handles all HDFS I/O, performs the Shuffle and Sort, and manages retries—the Python logic remains completely stateless and single-threaded.

8.2 Advantages of Python-based Streaming

1. **Rapid Prototyping:** Python's ecosystem (pandas, numpy, scikit-learn) allows complex feature extraction logic to be developed and unit-tested locally in minutes.
2. **No Compilation Required:** Unlike native Java MapReduce, Python scripts require no build step. Deployment is as simple as passing the script path to the `-file` Streaming flag.
3. **Library Richness:** Third-party libraries can be bundled into a Python virtual environment archive and distributed to all worker nodes via HDFS, making advanced analytics available across the cluster.

8.3 Local Simulation Architecture (`run_local.py`)

The local simulation replicates the Hadoop Streaming pipeline using Python `subprocess` and `shell` piping. The sequence is:

1. Read the input CSV file byte-stream.
2. Pipe contents through `mapper.py` via `Popen` with `stdout=PIPE`.

3. Sort the Mapper output lines lexicographically (simulating the Hadoop sort step).
4. Pipe the sorted intermediate stream into `reducer.py`.
5. Capture and display the final output.

This design ensures that any correctness validation performed locally will transfer directly to a production Hadoop cluster with no code modification, since the executable interface is identical.

9. Comparative Study Summary and Conclusions

9.1 Key Findings

This assignment successfully demonstrated the full MapReduce lifecycle across two datasets of vastly different scales:

- The **patient consent pipeline** correctly aggregated 18 patient records into per-category consent counts, revealing meaningful privacy preference gradients.
- The **healthcare pipeline** accurately extracted and counted six medical conditions across 55,500 rows, confirming the scalability of the identical Reducer logic.
- The **benchmark study** showed that execution time grew only 2.51× despite a 3,083× increase in record count, empirically validating the sub-linear scaling behaviour inherent to the MapReduce model.

9.2 Limitations and Future Work

The local simulation, while functionally correct, does not fully represent real Hadoop performance for two reasons:

1. **Single-threaded execution:** A real cluster would parallelize Map tasks across multiple nodes, reducing wall-clock time proportionally.
2. **In-memory sort:** Python sorts the full intermediate output in RAM. Hadoop performs an external multi-pass merge sort designed for datasets too large to fit in memory.

Future extensions of this work could deploy the identical scripts on a cloud-hosted Hadoop cluster (e.g., Amazon EMR or Google Dataproc) to observe true horizontal scaling behaviour and network I/O characteristics on multi-terabyte healthcare corpora.

9.3 Conclusion

MapReduce remains one of the most influential algorithmic paradigms in the history of distributed computing. By enforcing strict functional decomposition—Map transforms, Shuffle connects, Reduce aggregates—the model achieves fault tolerance, scalability, and

simplicity simultaneously. This assignment has validated these theoretical properties through empirical benchmarking on real healthcare data, providing a concrete foundation for understanding why Distributed Data Management systems built atop Hadoop have become indispensable infrastructure for modern clinical data analytics.